Performance Evaluation of Laravel Application Deployment on Docker

1st Abiyoga Dhaniswara
Department of Information System
Telkom University
Bandung, Indonesia
abiyogadhaniswara@student.telkomuniversity.ac.id

2nd Tien Fabrianti Kusumasari
Department of Information System
Telkom University
Bandung, Indonesia
tienkusumasari@telkomuniversity.ac.id

3rd Ekky Novriza Alam
Department of Information System
Telkom University
Bandung, Indonesia
ekkynovrizalam@telkomuniversity.ac.id

2025 International Conference on Information Technology and Computing (ICITCOM) | 979-8-3315-8525-9/25/$31.00 ©2025 IEEE | DOI: 10.1109/ICITCOM66635.2025.11265204

***Abstract*—This study evaluates the performance comparison of Laravel framework-based application deployment in a containerised environment using the traditional PHP-FPM + Nginx architecture and the modern integrated FrankenPHP architecture with Laravel Octane. A Laravel 12.x test application was deployed in Docker containers, and its performance was measured using Apache JMeter on two API endpoints representing different data retrieval workloads. The primary metrics analysed include average response time, throughput, and CPU and memory resource utilisation. The results indicate that on the endpoint with a large data processing load, both deployment architectures exhibited low performance and reached the capacity limits of computational resources, suggesting the primary bottleneck resides at the application level. However, on the endpoint with a lighter workload per request, FrankenPHP significantly outperformed PHP-FPM + Nginx regarding response time and throughput, supported by percentile analysis showing more stable performance under peak load. Nevertheless, FrankenPHP was recorded to have higher memory consumption. This study concludes that FrankenPHP offers potential performance improvements for specific workloads in Laravel applications on Docker, but it is essential to consider the trade-off with memory usage. These findings provide empirical insights for developers and engineers when selecting deployment architectures.**

Keywords—laravel, frankenphp, php-fpm, web server

I. Introduction

PHP remains one of the most dominant backend technologies for web application development. Such widespread popularity makes the performance aspects of PHP applications in production environments a primary concern [1]. Based on server-side technology usage data detected by W3Techs in March 2025, PHP is used by over 74% of websites with detectable server-side technology [2]. Large-scale web applications demand real-time responses and the ability to handle traffic surges, thus making deployment architecture and PHP code execution efficiency crucial [3].

Traditionally, PHP applications (including those based on the Laravel framework) are deployed using a separate architecture for the web server and the PHP executor. In this architecture, the web server communicates with the PHP interpreter to handle dynamic requests using APIs such as FastCGI [4]. This type of architecture has proven reliable and is widely used in production. Nevertheless, the inter-process communication overhead in traditional architectures like PHP-FPM+Nginx introduces significant latency that limits maximum performance potential, especially in large-scale applications [5], thus requiring alternative architectural approaches.

Laravel was chosen as the focus of study in this paper because it is one of the most popular and widely used PHP frameworks [6]. Empirical studies show that Laravel can achieve the highest throughput and better memory efficiency than other popular PHP frameworks [7]. Furthermore, in specific contexts, Laravel is recorded as having a faster average response time than Node.js (Express) framework for REST API (Representational State Transfer Application Programming Interface) services [8]. Although Laravel demonstrates good performance, its implementation in the PHP-FPM+Nginx architecture still inherits the previously mentioned inter-process latency. FrankenPHP emerges as an alternative approach to address this issue. FrankenPHP is a modern PHP application server built on top of the Caddy web server (based on Go) by integrating the PHP executor [9].

A comprehensive evaluation comparing the two approaches mentioned above is critical from both academic and practical perspectives. Academically, this study will contribute to understanding PHP runtime architectures and whether integrated models (such as FrankenPHP) are superior to traditional separate models in real applications (Laravel) and container environments. This is relevant considering that previous literature has focused mainly on optimization at the application or framework level, while studies concerning the PHP application server layer are still limited. The evaluation measures key performance metrics (such as response time, throughput, and resource usage) under specific workload scenarios. Thus, it is expected that a clear understanding of the advantages and trade-offs of each approach will be gained, which will be beneficial for both scientific advancement and practical application in the industry.

II. Literature Review

A. Traditional Model (Web Server + PHP-FPM)

The conventional architecture for PHP applications separates the roles of the web server and the PHP interpreter. The web server (such as Nginx or Apache) acts as a reverse proxy that receives HTTP requests, then forwards requests for PHP scripts to separate PHP execution processes via the FastCGI protocol. PHP-FPM (FastCGI Process Manager) is the primary implementation of FastCGI for PHP, featuring process pool management, logging, and support for high-load websites [10], [11], [12].

In this configuration, PHP-FPM runs a pool of PHP processes that listen on a port/socket and process requests sent by the web server [10]. This approach enhances flexibility and stability (as the web server and PHP run separately), but introduces inter-process communication overhead and application re-initialization on each request.

B. Integrated Model (FrankenPHP)

The integrated approach combines the functions of the web server and PHP runtime into a single, unified process or service. FrankenPHP is a modern PHP app server based on

Authorized licensed use limited to: Instituto Federal do Maranhao. Downloaded on October 23,2026 at 19:34:47 UTC from IEEE Xplore. Restrictions apply.

Caddy (a web server written in Go) that embeds the PHP interpreter within it [9]. Unlike the traditional stack, which requires separate services (Nginx + PHP-FPM), FrankenPHP is designed as a single binary that handles HTTP requests and executes PHP code in-process simultaneously [9]. With its 'worker mode,' FrankenPHP can bootstrap Laravel or Symfony applications once at the start and retain their state in memory, instead of reloading all components on every request [9]. According to the FrankenPHP documentation, this approach can significantly improve performance; internal benchmarks show throughput up to ~3.5 times higher than PHP-FPM on API Platform applications [9].

The literature on the direct comparison between traditional and integrated models is still limited, considering that FrankenPHP and similar technologies only emerged around 2022–2023. However, the basic concept is supported by general understanding: separate processes (like PHP-FPM) provide isolation but add IPC latency, whereas integrated processes reduce latency at the risk of potential centralized failure. Overall, integrated architectures like FrankenPHP offer a new paradigm promising higher performance and simpler configuration, although their production adoption is still in its early stages.

C. Laravel Framework Performance

As a feature-rich full-stack framework, Laravel is often considered to have greater overhead than simpler PHP frameworks. An empirical study by Li et al. [13] compared the performance of three PHP frameworks (Laravel, CodeIgniter, CakePHP) using the same application scenario. The results showed that CodeIgniter delivered the best performance across three test tasks, while Laravel excelled in one task but performed the worst in another [13]. In general, CodeIgniter (being a lighter framework) had the highest performance, while CakePHP had the lowest, and Laravel was in the middle (with variations depending on the case) [13]. These findings indicate that Laravel's feature-rich architecture can impact response time in some scenarios.

As Laravel evolved, its community recognized the need for performance improvements under high load. A significant improvement came with the introduction of Laravel Octane, which allows Laravel to run on application servers such as FrankenPHP, Swoole, or RoadRunner. Laravel Octane keeps the application in memory between requests, avoiding the re-initialization of Laravel components on every request. The official documentation notes that Octane "accelerates application performance" by leveraging high-speed servers, including FrankenPHP and Swoole [14]. This step marks a paradigm shift for Laravel, approaching an execution model similar to Node.js or long-running Java applications.

From the perspective of empirical studies in production environments, Chen et al. [15] reported on industry experience in addressing performance bottlenecks in Laravel web applications. Using static and dynamic analysis, they identified and refactored performance anti-patterns (e.g., issues with multiple queries in ORM), which successfully improved application efficiency [15]. This study confirms that the real-world performance of Laravel is influenced by deployment infrastructure, code quality, and database access patterns. With query optimization and caching, many of Laravel's performance limitations can be overcome at the application level. Nevertheless, optimization at the server architecture level (such as using FrankenPHP/Octane) remains crucial for reducing baseline overhead.

Overall, the literature indicates that Laravel can handle high production loads provided it is supported by appropriate configuration and optimization. Caching, database indexing, query optimization, and horizontal load balancing are standard practices for large-scale scenarios. Container deployment also facilitates scaling Laravel instances horizontally. However, the emerging gap is the lack of focused academic studies that quantitatively measure the performance comparison of Laravel deployment on traditional vs. integrated architectures.

D. Containerization of PHP Applications

Docker has become the de facto standard for distributing and running applications, including PHP/Laravel applications [16]. With Docker, the PHP runtime environment (extensions and dependencies) can be packaged in a consistent image, ensuring consistency between development and production environments [17]. Generally, there are two main approaches for deploying PHP applications using Docker. The first approach is a single container, multi-process, where the web server (e.g., Nginx or Apache) and the PHP interpreter are run together within a single container. The second approach is multi-container, where the PHP-FPM component and the Nginx web server are run in separate containers that communicate via Docker's internal network.

The recommended container practice tends to separate the primary process per container (following the "one process per container" principle), which is why the multi-container approach is frequently used [18].

Containerization introduces some overhead compared to bare-metal, but studies show this overhead is relatively small. Truyen et al. evaluated Docker's performance against native host for CPU-bound workloads and found that Docker host mode deployment introduced only "very small/insignificant" overhead compared to direct deployment without containers [19]. Additionally, Saputra and Subektiningsih [20] observed the performance of the Apache web server in Docker containers compared to a native environment. They found that at higher workloads, the performance difference in requests per second and transfer rates became relatively small, indicating that both environments exhibited similar performance when approaching resource limits. The average difference in total time results was only around 7% in their tests [20]. Findings like these reinforce that containerization overhead is generally not a primary factor degrading performance.

Regarding server architecture, modern trends like FrankenPHP can also be utilized within containers. An official FrankenPHP Docker image allows users to run Laravel in a single self-contained container with integrated Caddy+PHP. This is appealing because it can reduce the number of moving parts (without a separate Nginx container).

Authorized licensed use limited to: Instituto Federal do Maranhao. Downloaded on October 23,2026 at 19:34:47 UTC from IEEE Xplore. Restrictions apply.

E. Web Application Performance Testing

Researchers and practitioners utilize various load testing tools to evaluate deployment performance. Apache JMeter stands out as a widely used open-source web application load testing tool. JMeter can simulate many virtual users who send HTTP requests to the server in parallel, recording metrics such as throughput, response latency, and error rate. According to Lenka et al., "Apache JMeter is a web application performance testing tool useful for checking server performance under heavy load" [21].

JMeter is used in many research and industry tests. For instance, Alnuhait et al. [22] used Apache JMeter to test the responsiveness and throughput of two e-government websites (RSMD and E-GOV) under various load scenarios. The test results found significant performance degradation when the number of users and test duration increased [22].

In web application performance testing, throughput (number of requests per second), response time, and resource utilization are the main performance metrics measured [23]. These three metrics are crucial in evaluating web application deployment, as throughput reflects the system's capacity to handle load, response time indicates users' perceived speed of the service, and resource utilization shows the system's efficiency and stability under operational load.

JMeter can be operated with configured test scenarios (number of threads, ramp-up, loop, etc.) to identify weak points in web applications. Derezinska and Kwaśnik [24] highlight that load testing with JMeter can reveal workload issues in tested web applications. In that study, they gradually simulated 7000 users and measured average response time and percentiles (90%, 95%, 99%) as indicators of system stability [24]. JMeter test results are often presented as response time distributions (median, p90, p99) to assess whether bottlenecks occur.

Besides JMeter, there are other tools such as BlazeMeter (a cloud service compatible with JMeter scripts), Gatling, Locust, and Apache Bench. However, the reviewed literature tends to use JMeter due to its flexibility and support for the community. Indrianto [25] conducted a performance evaluation of a Laravel-based school information system through load testing using Apache JMeter and BlazeMeter. Testing was performed on several modules (login, profile update, image upload, etc.) with 50 to 100 simultaneous users. The report found that the system could handle the load with a stable average response time, throughput increasing with the number of users, and deviation decreasing [25]. Based on relevant studies, the community appears to have strong confidence in Apache JMeter as a tool for evaluating web-based system performance.

III. Research Method

This paper compares the performance of two Laravel application deployment architectures in a Docker container environment. To achieve this objective, an experiment was designed to test a sample application under load scenarios using Apache JMeter. Measurements were performed on key performance metrics such as response time, throughput, and computational resource utilization. Fig. 1 illustrates the flow of this research methodology, from environment preparation to data collection and analysis.

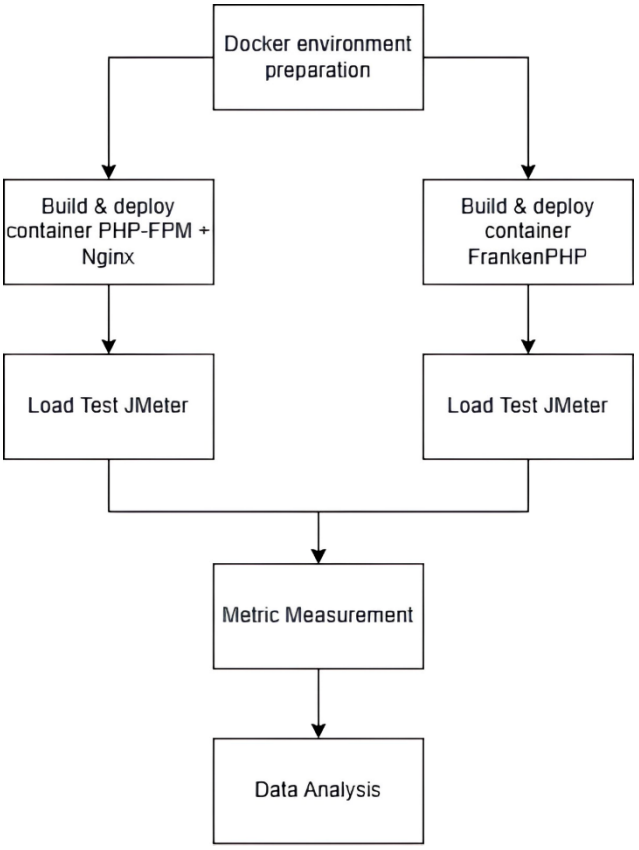


Fig. 1. Flow of Experiment Methodology

This performance comparison experiment was conducted in a controlled environment using Docker to create consistent and isolated conditions for each test. The testing infrastructure consists of two application deployment architectures prepared using Docker Compose.

In the first architecture, Nginx and PHP-FPM are run together within a single container. This container is built from the official php:8.2-fpm image, with Nginx installed inside it. Supervisor manages both processes, where Nginx forwards PHP requests to PHP-FPM via a TCP socket at 127.0.0.1:9000. PHP-FPM is configured in static mode with pm.max_children=5, ensuring five child processes are always available. Each process is automatically restarted after handling every 500 requests (pm.max_requests = 500) to prevent potential memory leaks.

In the second architecture, FrankenPHP (using the official dunglas/frankenphp:php8.2 image) integrates the Caddy web server and PHP runtime. The Laravel application is run on top of Laravel Octane using the command php artisan octane:start --server=frankenphp --workers=5 --max-requests=500. Each Octane worker is restarted after handling 500 requests, mimicking the same pattern as PHP-FPM in the first scheme and ensuring consistent execution conditions.

To support the Laravel application, two separate MySQL 8.0 database containers were prepared (one for each deployment scheme). Both database containers were ensured to have identical initial data imported from the same source. This ensures that differences in performance metrics are not

Authorized licensed use limited to: Instituto Federal do Maranhao. Downloaded on October 23,2026 at 19:34:47 UTC from IEEE Xplore. Restrictions apply.

caused by data variation. Computational resource allocation was also kept uniform for both tested application architectures. The total resource limits allocated for each architecture were equalized using the deploy.resources.limits configuration in the docker-compose.yml file and limit it to 4 vCPU and 2 GB RAM.

The core PHP configuration was made uniform. The PHP OPcache module was enabled in both with the same configuration (OPcache memory allocation of 128 MB, opcache.validate_timestamps=0 to disable script update checking, and JIT disabled with opcache.jit=0).

Apache JMeter 5.6.3 was used as the load testing tool to measure the performance of both architectures. JMeter was chosen because it can simulate many virtual users in parallel and accurately measure performance metrics. The test scenario was designed as a Load Test to examine the system's behavior under expected normal load.

The Load Test scenario was configured with 100 threads (virtual users) in JMeter, a 5-second ramp-up period, and a 600-second (10-minute) test duration. A load of 100 concurrent users represents the expected peak load still within the system's capacity limits.

The main performance metrics analyzed are average response time and throughput. Response time measures the duration the system requires to respond to an HTTP request, while throughput is defined as the number of completed requests divided by the total test duration. Throughput is expressed in requests per second (requests/s). The values of these metrics are extracted from JMeter test results reports for each test scenario. In addition to application metrics, CPU and memory utilization for each container were also measured. CPU utilization indicates the percentage of CPU time used to process requests, while memory utilization shows the proportion of memory consumed.

The Laravel application used in this testing is a thesis defense management application based on the Course Learning Outcome (CLO) module, developed using Laravel 12.x and PHP 8.2. This application includes CLO features relevant for performance testing and was implemented without changes to the business logic on both compared server architectures.

500 CLO data entries were prepared in the database for each test and imported from the same source for both scenarios to ensure uniform test data. The CLO data includes information relevant to the application's endpoints, and the volume of 500 entries was chosen to be sufficiently representative for load simulation.

The Load Test focused on the following two API endpoints: (1) GET /api/clos, which retrieves a list of all CLO entries, and (2) GET /api/clos/preview/1/1/pembimbing, which retrieves specific preview details related to CLO data. These two endpoints were tested separately to observe how each function responds to different loads.

IV. Result and Evaluations

A. API Endpoint Performance Evaluation

Performance testing was conducted on two primary API endpoints of the thesis defense management application, namely /api/clos and /api/clos/preview/1/1/pembimbing. Both endpoints were tested using a load test scenario (100 virtual users, 5-second ramp-up, 10-minute duration) with the following compared deployment architectures: (i) PHP-FPM + Nginx in a single container, and (ii) FrankenPHP as an integrated server.

Table 1 presents the average response time and throughput test results for both endpoints. Testing was conducted with Apache JMeter, and data was collected during the experiment under equivalent resource conditions (4 CPU cores and 2.5 GB RAM for each scenario).

Table 1. Average Response Time and Throughput Results

Endpoint	Deployment	Samples	Avg Response Time (ms)	Throughput (req/s)
/api/clos	PHP-FPM + Nginx	150	25,251	2.4
/api/clos	FrankenPHP (Worker Mode)	140	25,870	2.3
/api/clos/preview/1/1/pembimbing	PHP-FPM + Nginx	118,035	506	196.5
/api/clos/preview/1/1/pembimbing	FrankenPHP (Worker Mode)	139,979	426	233.1

Both architectures showed very low performance for the /api/clos endpoint with extreme response times (>25 seconds) and throughput of only 2.3–2.4 req/s. The throughput difference is minor and likely due to natural fluctuations during testing. Furthermore, the high response time is caused by the large volume of data exceeding buffer capacity. This indicates a capacity limit for the architectures when reading large amounts of data. For more details regarding the response time distribution for this endpoint, please refer to Fig. 2.

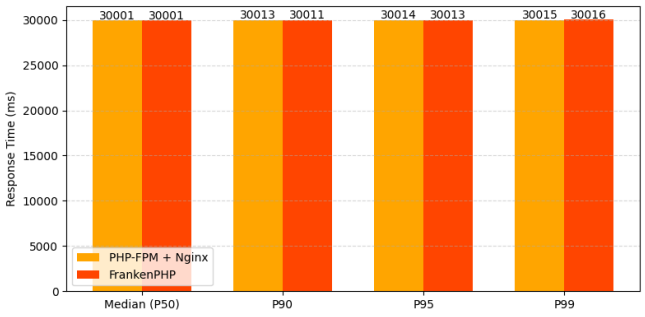


Fig. 2. Response Time Distribution (Percentile) for Endpoint /api/clos

It is clear from Fig. 2 that the response times at all measured percentiles, from median (P50) up to the 99th percentile (P99), show very high values, ranging from 30001 ms to 30016 ms. A comparison between PHP-FPM + Nginx and FrankenPHP shows an almost identical response time pattern across the entire percentile range. This consistently

Authorized licensed use limited to: Instituto Federal do Maranhao. Downloaded on October 23,2026 at 19:34:47 UTC from IEEE Xplore. Restrictions apply.

high response time across all percentiles reinforces the previous conclusion from Table 1 regarding the very low performance of the /api/clos endpoint. This indicates that the bottleneck encountered at this endpoint not only occurs for a small fraction of requests but also affects the majority of requests, resulting in an inferior user experience under the simulated load. This pattern further confirms that deployment architecture alone cannot overcome the significant inefficiency caused by processing large volumes of data on this endpoint.

The very poor performance of the /api/clos endpoint is caused by an application-level bottleneck rather than the server architecture. Analysis of the source code reveals that the indexApi method handling this endpoint retrieves all data from the CLO table along with its associated relationships (CLO::with(['components.intervals'])->get()). This operation fetches 500 CLO records with all related data in a single request, resulting in an excessively large data object. The serialization of this large object into JSON format significantly consumes CPU and memory resources, thus causing very high response times and low throughput.

For the /api/clos/preview/1/1/pembimbing endpoint, FrankenPHP recorded an average response time of approximately 15.8% faster and throughput 18.6% higher than PHP-FPM + Nginx. This indicates the efficiency of the event-driven architecture adopted by FrankenPHP, which minimizes inter-process communication overhead typical of conventional FastCGI architectures. A more detailed analysis of the response time distribution can be seen in Fig. 3.

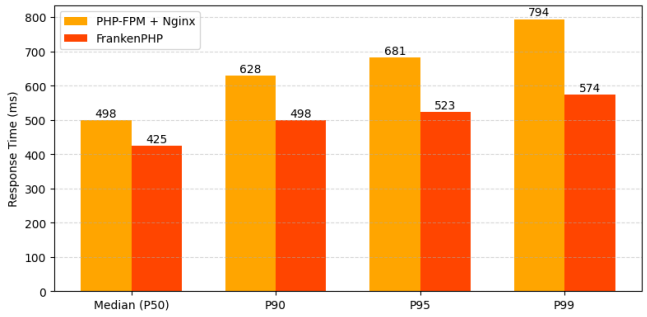


Fig. 3. Response Time Distribution (Percentile) for Endpoint /api/clos/preview/1/1pembimbing

Fig. 3 illustrates the response time distribution for the /api/clos/preview/1/1/pembimbing endpoint. Unlike the previous endpoint, FrankenPHP (Worker Mode) consistently shows lower response times at all percentile levels compared to the PHP-FPM + Nginx configuration for this workload. For example, at the median (P50), FrankenPHP response time was 425 ms, while PHP-FPM + Nginx was 498 ms. This difference is even more significant at higher percentiles, such as P99, where FrankenPHP response time was 574 ms, substantially lower than 794 ms for PHP-FPM + Nginx. This pattern indicates that the advantage of FrankenPHP is not limited to average response time but results in more stable and responsive performance under peak load. The significantly lower P99 response time for FrankenPHP indicates that even handling the slowest requests is more efficient than the conventional architecture. These findings support the hypothesis that the reduced inter-process communication overhead in the integrated architecture has a measurable positive impact on performance, especially for endpoints with relatively lighter data processing.

B. *Evaluation of Resource Utilization*

During testing, CPU and memory usage for each container were observed via Docker Stats. Visualizations of resource consumption over time are presented in Fig. 4, 5, 6, and 7 to illustrate detailed resource utilization patterns.

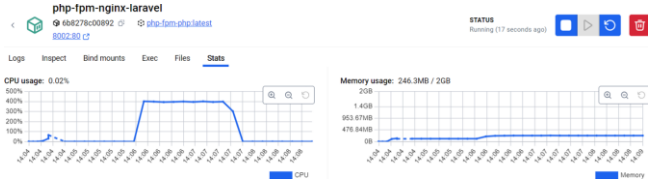


Fig 4. CPU & Memory Utilization (PHP-FPM + Nginx) for the /api/clos Endpoint

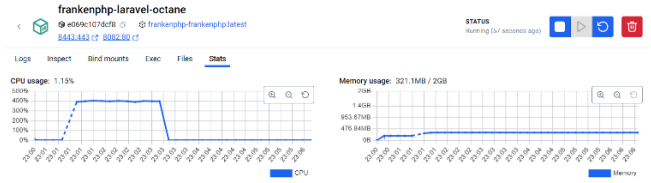


Fig. 5. CPU & Memory Utilization (FrankenPHP) for the /api/clos Endpoint

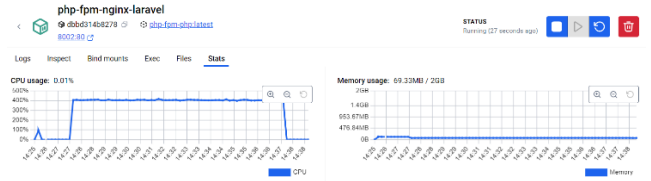


Fig. 6. CPU & Memory Utilization (PHP-FPM + Nginx) for the /api/clos/preview/1/1pembimbing Endpoint

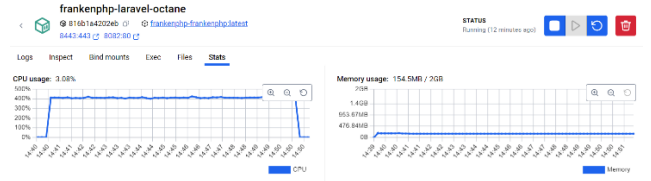


Fig. 7. CPU & Memory Utilization (FrankenPHP) for the /api/clos/preview/1/1pembimbing Endpoint

Based on Fig. 4, 5, 6, and 7, both deployment architectures for the /api/clos and /api/clos/preview/1/1/pembimbing endpoints showed CPU usage near 400%, indicating that all four cores were operating at maximum capacity during the testing. Memory usage showed a significant difference: for the /api/clos endpoint, PHP-FPM+Nginx used approximately 246 MB, while FrankenPHP used approximately 321 MB. For the /api/clos/preview/1/1/pembimbing endpoint, PHP-FPM+Nginx memory usage decreased to about 69 MB, while FrankenPHP still utilized around 155 MB to run the integrated PHP runtime with the web server. This difference confirms the presence of a trade-off between performance and resource efficiency. FrankenPHP provides better responsiveness with higher memory overhead, whereas the conventional architecture is more memory-efficient but has lower throughput.

V. Conclusion

This research compared the performance of Laravel application deployment in Docker containers using two different architectures: traditional PHP-FPM + Nginx and

Authorized licensed use limited to: Instituto Federal do Maranhao. Downloaded on October 23,2026 at 19:34:47 UTC from IEEE Xplore. Restrictions apply.

modern FrankenPHP with Laravel Octane. The test results show that performance is highly dependent on workload characteristics. Both architectures showed poor performance for the endpoint processing large volumes of data (/api/clos). They reached CPU capacity limits, indicating the bottleneck occurred at the application or data processing level, rather than primarily at the server architecture level. However, for the endpoint with a lighter per-request load (/api/clos/preview), FrankenPHP significantly provided faster response times, higher throughput, and better stability under peak load compared to PHP-FPM + Nginx. This advantage is attributed to FrankenPHP's long-running execution model, which reduces inter-process communication overhead.

On the other hand, the evaluation of resource utilization shows that FrankenPHP generally requires a larger memory allocation. This indicates a trade-off between the potential performance improvement offered by FrankenPHP for specific workloads and its higher memory consumption. In conclusion, the choice of deployment architecture between PHP-FPM + Nginx and FrankenPHP needs to consider the specific characteristics of the application workload and the priority between absolute performance and memory usage efficiency. This study contributes by providing empirical data for this comparison, while future research can explore application-level optimizations and testing at a larger scale.

REFERENCES

[1] A. Niarman, Iswandi, and A. K. Candri, "Comparative Analysis of PHP Frameworks for Development of Academic Information System Using Load and Stress Testing," Int. J. Softw. Eng. Comput. Sci. IJSECS, vol. 3, no. 3, pp. 424–436, Dec. 2023, doi: 10.35870/ijsecs.v3i3.1850.

[2] W3Techs, "W3Techs - extensive and reliable web technology surveys." Accessed: Apr. 24, 2025. [Online]. Available: https://w3techs.com/

[3] J. Dean and L. A. Barroso, "The tail at scale," Commun. ACM, vol. 56, no. 2, pp. 74–80, Feb. 2013, doi: 10.1145/2408776.2408794.

[4] A. Bulekov, R. Jahanshahi, and M. Egele, "Saphire: Sandboxing PHP Applications with Tailored System Call Allowlists," May 2025.

[5] G. Yeung, "Reducing Data Copying Overhead in Web Servers," 2010.

[6] M. Penelova, "Hybrid Role and Attribute Based Access Control Applied in Information Systems," Cybern. Inf. Technol., vol. 21, no. 3, pp. 85–96, Sep. 2021, doi: 10.2478/cait-2021-0031.

[7] M. Laaziri, K. Benmoussa, S. Khoulji, and M. L. Kerkeb, "A Comparative study of PHP frameworks performance," Procedia Manuf., vol. 32, pp. 864–871, 2019, doi: 10.1016/j.promfg.2019.02.295.

[8] D. Gope, D. J. Schlais, and M. H. Lipasti, "Architectural Support for Server-Side PHP Processing," in Proceedings of the 44th Annual International Symposium on Computer Architecture, Toronto ON Canada: ACM, Jun. 2017, pp. 507–520. doi: 10.1145/3079856.3080234.

[9] "FrankenPHP: The Modern PHP App Server." Accessed: Apr. 24, 2025. [Online]. Available: https://frankenphp.dev/docs/

[10] "PHP: FastCGI Process Manager (FPM) - Manual." Accessed: May 01, 2025. [Online]. Available: https://www.php.net/manual/en/install.fpm.php

[11] "Using nginx as HTTP load balancer." Accessed: May 01, 2025. [Online]. Available: https://nginx.org/en/docs/http/load_balancing.html

[12] L. M. Sequeira, L. Morgado, and E. J. S. Pires, "Simplifying Crowd Automation in the Virtual Laboratory of Archaeology," Procedia Technol., vol. 13, pp. 56–65, 2014, doi: 10.1016/j.protcy.2014.02.009.

[13] X. Li, S. Karnan, and J. A. Chishti, "An empirical study of three PHP frameworks," in 2017 4th International Conference on Systems and Informatics (ICSAI), Hangzhou: IEEE, Nov. 2017, pp. 1636–1640. doi: 10.1109/icsai.2017.8248546.

[14] Laravel, "Laravel Octane - Laravel 12.x - The PHP Framework For Web Artisans." Accessed: Apr. 25, 2025. [Online]. Available: https://laravel.com/

[15] B. Chen, Z. M. Jiang, P. Matos, and M. Lacaria, "An Industrial Experience Report on Performance-Aware Refactoring on a Database-Centric Web Application," in 2019 34th IEEE/ACM International Conference on Automated Software Engineering (ASE), San Diego, CA, USA: IEEE, Nov. 2019. doi: 10.1109/ase.2019.00066.

[16] G. Rosa, F. Zappone, S. Scalabrino, and R. Oliveto, "Fixing Dockerfile smells: an empirical study," Empir. Softw. Eng., vol. 29, no. 5, Sep. 2024, doi: 10.1007/s10664-024-10471-7.

[17] Docker, "Why Docker Compose?," Docker Documentation. Accessed: Apr. 25, 2025. [Online]. Available: https://docs.docker.com/guides/docker-compose/why/

[18] "Run multiple processes in a container | Docker Docs." Accessed: May 02, 2025. [Online]. Available: https://docs.docker.com/engine/containers/multi-service_container/

[19] E. Truyen, D. Van Landuyt, B. Lagaisse, and W. Joosen, "Performance overhead of container orchestration frameworks for management of multi-tenant database deployments," in Proceedings of the 34th ACM/SIGAPP Symposium on Applied Computing, Limassol Cyprus: ACM, Apr. 2019, pp. 156–159. doi: 10.1145/3297280.3297536.

[20] R. B. Saputra and S. Subektiningsih, "Comparative Analysis of Apache 2 Performance in Docker Containers vs Native Environment," J. Ilm. Tek. Elektro Komput. Dan Inform., vol. 9, no. 4, pp. 1024–1034, Nov. 2023, doi: 10.26555/jiteki.v9i4.27220.

[21] Rakesh Kumar Lenka, Meenu Rani Dey, Pranali Bhanse, and Rabindra Kumar Barik, "Performance and Load Testing: Tools and Challenges," presented at the 2018 International Conference on Recent Innovations in Electrical, Electronics & Communication Engineering (ICRIEECE), Bhubaneswar, India: IEEE, 2018. doi: 10.1109/icrieece44171.2018.

[22] H. Alnuhait, W. Alzyadat, A. Althunibat, H. Kahtan, B. Zaqaibeh, and H. A. Al-Khawaja, "Web application performance assessment: A study of responsiveness, throughput, and scalability," Int. J. Adv. Appl. Sci., vol. 11, no. 9, pp. 214–226, Sep. 2024, doi: 10.21833/ijaas.2024.09.023.

[23] M. Bolanowski, M. Ćmil, and A. Starzec, "New Model for Defining and Implementing Performance Tests," Future Internet, vol. 16, no. 10, p. 366, Oct. 2024, doi: 10.3390/fi16100366.

[24] A. Derezinska and K. Kwaśnik, "Performance-based Refactoring of Web Application: A Case of Public Transport," in Proceedings of the 15th International Conference on Evaluation of Novel Approaches to Software Engineering, Prague, Czech Republic: SCITEPRESS - Science and Technology Publications, 2020, pp. 611–618. doi: 10.5220/0009797006110618.

[25] I. Indrianto, "PERFORMANCE TESTING ON WEB INFORMATION SYSTEM USING APACHE JMETER AND BLAZEMETER," J. Ilm. Ilmu Terap. Univ. Jambi, vol. 7, no. 2, pp. 138–149, Dec. 2023, doi: 10.22437/jiituj.v7i2.2844J. Clerk Maxwell, A Treatise on Electricity and Magnetism, 3rd ed., vol. 2. Oxford: Clarendon, 1892, pp.68–73.

Authorized licensed use limited to: Instituto Federal do Maranhao. Downloaded on October 23,2026 at 19:34:47 UTC from IEEE Xplore. Restrictions apply.